

PROBLEM SOLVING AND TESTING USING JAVA

WEEK 1

Java Lambda Expressions

```
import java.util.*;
interface PerformOperation {
    boolean check(int n);
}
public class Main {
    static PerformOperation isOdd() {
        return n -> n % 2 != 0;
    }
    static PerformOperation isPrime() {
        return n -> {
            if (n < 2) return false;
            for (int i = 2; i * i <= n; i++)
                if (n % i == 0) return false;
            return true;
        };
    }
    static PerformOperation isPalindrome() {
        return n -> {
            int x = n, rev = 0;
            while (n > 0) {
                rev = rev * 10 + n % 10;
                n /= 10;
            }
            return x == rev;
        };
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int t = sc.nextInt();
        while (t-- > 0) {
            int type = sc.nextInt();
            int n = sc.nextInt();
            if (type == 1)
                System.out.println(isOdd().check(n) ? "ODD" : "EVEN");
            else if (type == 2)
                System.out.println(isPrime().check(n) ? "PRIME" : "COMPOSITE");
            else
                System.out.println(isPalindrome().check(n) ? "PALINDROME" : "NOT PALINDROME");
        }
    }
}
```

Microsoft Windows [Version 10.0.26200.8875]
(c) Microsoft Corporation. All rights reserved.

D:\>javac Main.java

D:\>java Main

5
1 4
EVEN
2 5
PRIME
3 898
PALINDROME
1 3
ODD
2 12
COMPOSITE

D:\>|

Java Comparator

```
import java.util.*;
class Player {
    String name;
    int score;
    Player(String name, int score) {
        this.name = name;
        this.score = score;
    }
}
class Checker implements Comparator<Player> {
    public int compare(Player a, Player b) {
        if (a.score != b.score)
            return b.score - a.score;
        return a.name.compareTo(b.name);
    }
}
public class Sample {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        Player[] players = new Player[n];
        for (int i = 0; i < n; i++) {
            String name = sc.next();
            int score = sc.nextInt();
            players[i] = new Player(name, score);
        }
        Arrays.sort(players, new Checker());
        for (Player p : players) {
            System.out.println(p.name + " " + p.score);
        }
        sc.close();
    }
}
```

```
C:\Windows\System32\cmd.e  X + v
Microsoft Windows [Version 10.0.26200.8875]
(c) Microsoft Corporation. All rights reserved.

D:\>javac Sample.java

D:\>java Sample
5
amy 100
david 100
heraldo 50
aakansha 75
aleksa 150
aleksa 150
amy 100
david 100
aakansha 75
heraldo 50

D:\>|
```

Java Sort

```
import java.util.*;
class Student {
    int id;
    String name;
    double cgpa;
    Student(int id, String name, double cgpa) {
        this.id = id;
        this.name = name;
        this.cgpa = cgpa;
    }
}
class Checker implements Comparator<Student> {
    public int compare(Student a, Student b) {
        if (a.cgpa != b.cgpa)
            return Double.compare(b.cgpa, a.cgpa);
        if (a.name.equals(b.name))
            return a.name.compareTo(b.name);
        return Integer.compare(a.id, b.id);
    }
}
public class Sort {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        Student[] students = new Student[n];
        for (int i = 0; i < n; i++) {
            int id = sc.nextInt();
            String name = sc.next();
            double cgpa = sc.nextDouble();
            students[i] = new Student(id, name, cgpa);
        }
        Arrays.sort(students, new Checker());
        for (Student s : students) {
            System.out.println(s.name);
        }
        sc.close();
    }
}
```

```
C:\Windows\System32\cmd.e x + v
Microsoft Windows [Version 10.0.26200.8875]
(c) Microsoft Corporation. All rights reserved.
```

```
D:\>javac Sort.java
```

```
D:\>java Sort
5
33 rumpa 3.68
85 ashis 3.85
56 samiha 3.75
19 samara 3.75
22 fahim 3.76
ashis
fahim
samara
samiha
rumpa
```

```
D:\>|
```

Running Sum of 1D Array

1480. Running Sum of 1d Array

Given an array `nums`. We define a running sum of an array as `runningSum[i] = sum(nums[0..i])`.

Return the running sum of `nums`.

Example 1:

Input: `nums = [1,2,3,4]`
Output: `[1,3,6,10]`
Explanation: Running sum is obtained as follows: `[1, 1+2, 1+2+3, 1+2+3+4]`.

Example 2:

Input: `nums = [1,1,1,1,1]`
Output: `[1,2,3,4,5]`
Explanation: Running sum is obtained as follows: `[1, 1+1, 1+1+1, 1+1+1+1, 1+1+1+1+1]`.

Example 3:

Input: `nums = [3,1,2,10,1]`
Output: `[3,4,6,16,17]`

Constraints:

- `1 <= nums.length <= 1000`
- `-106 <= nums[i] <= 106`

```
class Solution {
    public int[] runningSum(int[] nums) {
        for (int i = 1; i < nums.length; i++) {
            nums[i] = nums[i] + nums[i - 1];
        }
        return nums;
    }
}
```

Testcase 1: Test Result

Accepted Runtime: 0 ms

Case 1 **Case 2** **Case 3**

Input:

`nums =`
`[1,2,3,4]`

Output:

`[1,3,6,10]`

Expected:

`[1,3,6,10]`

Richest Customer Wealth

The screenshot shows the LeetCode interface for problem 1672, "Richest Customer Wealth". The left pane contains the problem description, examples, and constraints. The right pane shows a Java solution and the test results.

1672. Richest Customer Wealth

You are given an $n \times m$ integer grid `accounts` where `accounts[i][j]` is the amount of money the i^{th} customer has in the j^{th} bank. Return the *wealth* that the *richest* customer has.

A customer's **wealth** is the amount of money they have in all their bank accounts. The richest customer is the customer that has the maximum wealth.

Example 1:

Input: `accounts = [[1,2,3],[3,2,1]]`
Output: 6
Explanation:
1st customer has wealth = 1 + 2 + 3 = 6
2nd customer has wealth = 3 + 2 + 1 = 6
Both customers are considered the richest with a wealth of 6 each, so return 6.

Example 2:

Input: `accounts = [[1,5],[7,3],[3,5]]`
Output: 10
Explanation:
1st customer has wealth = 6
2nd customer has wealth = 10
3rd customer has wealth = 8
The 2nd customer is the richest with a wealth of 10.

Example 3:

Input: `accounts = [[2,8,7],[7,1,3],[1,9,5]]`
Output: 17

Constraints:

- $n == \text{accounts.length}$

```
class Solution {
    public int maximumWealth(int[][] accounts) {
        int maxWealth = 0;

        for (int[] customer : accounts) {
            int wealth = 0;
            for (int money : customer) {
                wealth += money;
            }
            maxWealth = Math.max(maxWealth, wealth);
        }

        return maxWealth;
    }
}
```

Testcase: Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input: `accounts = [[1,2,3],[3,2,1]]`

Output: 6

Expected: 6

Squares of a Sorted Array

The screenshot shows the LeetCode interface for problem 977, "Squares of a Sorted Array". The left pane contains the problem description, examples, and constraints. The right pane shows a Java solution and the test results.

977. Squares of a Sorted Array

Given an integer array `nums` sorted in **non-decreasing order**, return an array of *the squares of each number sorted in non-decreasing order*.

Example 1:

Input: `nums = [-4,-1,0,3,10]`
Output: `[0,1,9,16,100]`
Explanation: After squaring, the array becomes `[16,1,0,9,100]`. After sorting, it becomes `[0,1,9,16,100]`.

Example 2:

Input: `nums = [-7,-3,2,3,11]`
Output: `[4,9,9,49,121]`

Constraints:

- $1 \leq \text{nums.length} \leq 10^4$
- $-10^4 \leq \text{nums}[i] \leq 10^4$
- `nums` is sorted in **non-decreasing order**.

Follow up: Squaring each element and sorting the new array is very trivial, could you find an $O(n)$ solution using a different approach?

Discover more Computer Science

Solve this question in a real interview before? 1/6

```
class Solution {
    public int[] sortedSquares(int[] nums) {
        int n = nums.length;
        int[] result = new int[n];

        int left = 0;
        int right = n - 1;
        int index = n - 1;

        while (left <= right) {
            int leftSquare = nums[left] * nums[left];
            int rightSquare = nums[right] * nums[right];

            if (leftSquare > rightSquare) {
                result[index] = leftSquare;
                left++;
            } else {
                result[index] = rightSquare;
                right--;
            }

            index--;
        }

        return result;
    }
}
```

Testcase: Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input: `nums = [-4,-1,0,3,10]`

Output: `[0,1,9,16,100]`

Expected: `[0,1,9,16,100]`

Find Pivot Index

724. Find Pivot Index

Easy Topics Compare Hint

Given an array of integers `nums`, calculate the **pivot index** of this array.

The **pivot index** is the index where the sum of all the numbers **strictly** to the left of the index is equal to the sum of all the numbers **strictly** to the index's right.

If the index is on the left edge of the array, then the left sum is `0` because there are no elements to the left. This also applies to the right edge of the array.

Return the **leftmost pivot index**. If no such index exists, return `-1`.

Example 1:

Input: `nums = [1,7,3,6,5,6]`
Output: `3`
Explanation:
The pivot index is 3.
Left sum = `nums[0] + nums[1] + nums[2] = 1 + 7 + 3 = 11`
Right sum = `nums[4] + nums[5] = 5 + 6 = 11`

Example 2:

Input: `nums = [1,2,3]`
Output: `-1`
Explanation:
There is no index that satisfies the conditions in the problem statement.

Example 3:

Input: `nums = [2,1,-1]`
Output: `0`
Explanation:
The pivot index is 0.
Left sum = `0` (no elements to the left of index 0)
Right sum = `nums[1] + nums[2] = 1 + -1 = 0`

3.5K 277 119 Online

Code

Java Auto

```
3 int totalSum = 0;
4
5 // Calculate total sum
6 for (int num : nums) {
7     totalSum += num;
8 }
9
10 int leftSum = 0;
11
12 // Find pivot index
13 for (int i = 0; i < nums.length; i++) {
14     int rightSum = totalSum - leftSum - nums[i];
15
16     if (leftSum == rightSum) {
17         return i;
18     }
19     leftSum += nums[i];
20 }
21
22 return -1;
```

Send Ln 25, Col 1

Testcase Test Result

Input

nums =
[1,7,3,6,5,6]

Output

3

Expected

3

Contribute a testcase